

Studienarbeit

Entwicklung eines CAN-FD Datenloggers

Name	Leif Stechmann
Matrikelnummer	940786
Datum	30.07.2026
Betreuer	Achim Boll

Inhaltsverzeichnis

1. Einleitung	3
2. Methodik	3
2.1. Anforderungsanalyse	5
2.2. Anforderungsmatrix	5
2.3. Stand der Technik	7
2.3.1. Influx ReXgen Air	8
2.3.2. Kvaser Memorator Pro 5xHS	9
2.3.3. Vector GL2400	10
2.4. Referenzdesign-Analyse	11
2.5. Ziele und Umfang	15
2.6. Meilensteinplanung	16
3. Systemarchitektur	17
3.1. Grundarchitektur	17
3.1.1. Mikrocontroller	17
3.1.2. Externe CAN-FD-Controller	17
3.1.3. microSD-Karte	17
3.1.4. USB-Schnittstelle und Platine	17
3.1.5. Sender	18
3.2. Auswahl des Mikrocontrollers	18
3.3. Auswahl eines CAN- FD Controller-Transcievers	21
3.4. Auswahl eines Speichermediums	23
3.5. Blockdiagramm	26
3.6. Blockdiagramm des Prototypen	27
4. Senden von CAN-FD Nachrichten	27
5. Empfangen von CAN-FD-Nachrichten	28
5.1. Der Treiber des MCP2518FD	28
5.1.1. Serial Peripheral Interface	28
5.1.2. Registerzugriff über SPI	28
5.1.3. Konfiguration des Empfangspfads	30
5.1.4. Inbetriebnahme durch den Mikrocontroller	31
5.1.5. Interruptgesteuertes Auslesen	32
6. Literaturverzeichnis	34
7. Tabellenverzeichnis	36
8. Abbildungsverzeichnis	37

1. Einleitung

Moderne technische Systeme stehen unter dem Anspruch, maximale Qualität in Funktionalität, Energieeffizienz, Zuverlässigkeit sowie Wirtschaftlichkeit zu vereinen.

Die Grundlage jeder Entwicklung bilden mathematische Modelle, mit denen das Verhalten eines Systems bereits in der Entwurfsphase beschrieben und bewertet wird. Diese Modelle stellen jedoch zwangsläufig eine Vereinfachung der realen physikalischen Zusammenhänge dar. Ihre Aussagekraft ist erst dann gegeben, wenn sie durch Messdaten unter realen Betriebsbedingungen bestätigt werden.

Die präzise und zuverlässige Erfassung von Messdaten ist deshalb ein zentraler Bestandteil moderner Entwicklungsprozesse - sowohl in der industriellen Produktentwicklung als auch in der Formula Student. Sie ermöglicht es, theoretische Annahmen mit der Realität abzugleichen, konstruktive Schwachstellen zu identifizieren und Systeme gezielt weiterzuentwickeln.

Ziel dieser Studienarbeit ist deshalb die Erarbeitung eines Konzeptes zur Aufzeichnung und Analyse relevanter Fahrzeugdaten eines Formula- Student Rennwagens, sowie die Erforschung der technischen Grundlagen dieser.



Abbildung 1: Der A-25 von Raceyard. [1]

2. Methodik

Zur Erarbeitung eines funktionalen Konzeptes orientiert sich diese Studienarbeit an einer leicht abgewandelten Version des V- Modells, welches in der VDI- Richtlinie VDI/ VDE- 2206 beschrieben wird. [2]

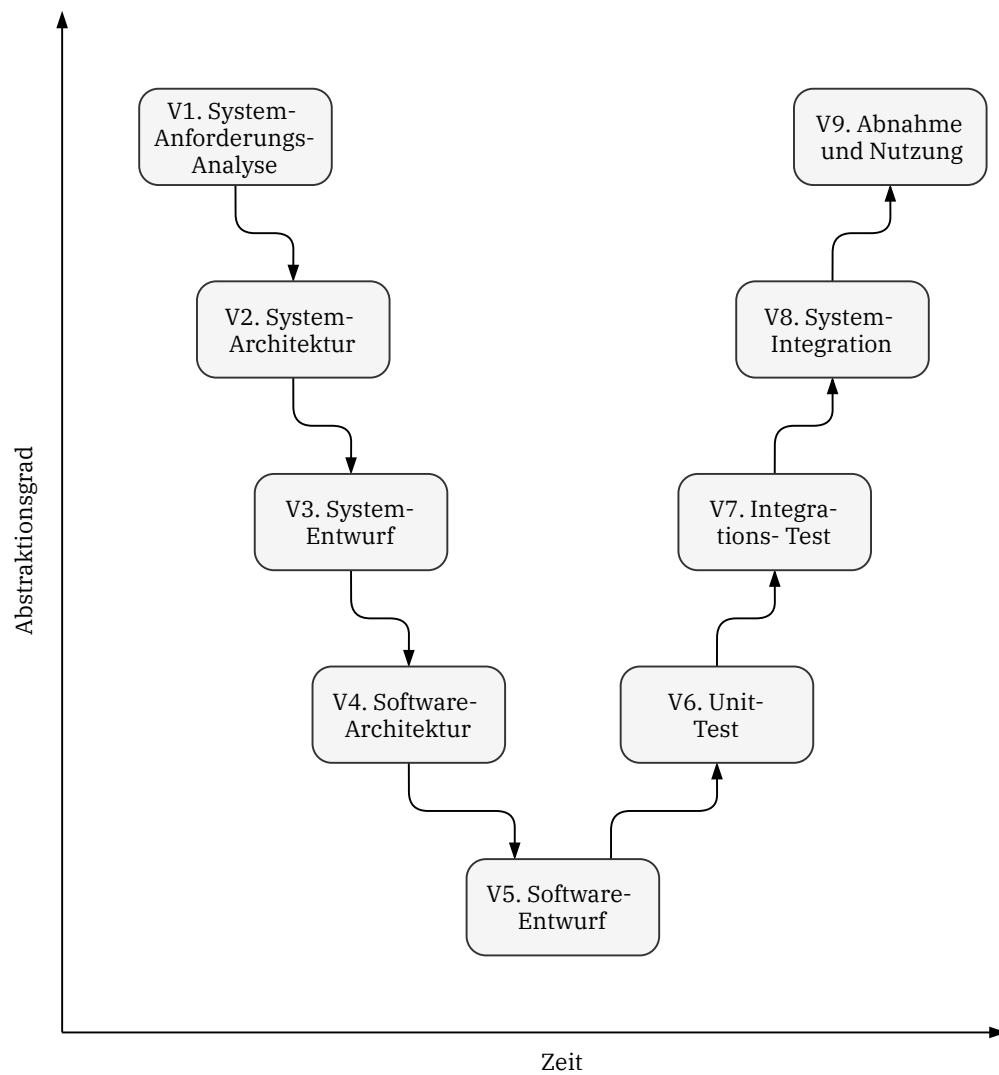


Abbildung 2: V-Modell. [2]

Dabei umfasst diese Studienarbeit jediglich Punkte V1 bis V5, sowie einen Unit- Test in stark vereinfachtem Umfang, um gezielt Fachwissen über die technischen Kernprinzipien aufzubauen.

2.1. Anforderungsanalyse

Der Datenlogger bildet eine wichtige Schnittstelle zwischen Mensch und Maschine. Er dient als zentrales Werkzeug für Ingenieure zur Erfassung und Auswertung von Fahrzeugdaten, um Konstruktionen zu Evaluieren und iterativ zu verbessern. Aus diesem Grund werden folgende Bedarfe des Endnutzers als Startpunkt für die Anforderungsanalyse verwendet:

- **A1: Einfache Nutzbarkeit**

Messdaten tragen nur dann zur tatsächlichen Verbesserung des Fahrzeuges bei, wenn sie schnell, einfach und ohne besondere Vorkenntnisse ausgewertet werden können. Der Ausleseprozess muss deshalb extrem simpel sein.

- **A2: Zuverlässige Datenerfassung**

Wichtige Informationen dürfen nicht verloren gehen. Die Messdaten müssen auch bei hohen Datenmengen des Fahrzeugs und unmittelbar vor einem Fehler oder vollständigen Systemausfall verlässlich verfügbar sein.

- **A3: Zeit- und Ressourceneffiziente Herstellung.**

In den meisten modernen Industriezweigen sowie auch in der Formula Student sind die Produktentwicklungszyklen extrem schnell. Das Team von Raceyard muss innerhalb nur eines halben Jahres einen Rennwagen konstruieren und fertigen. Entwicklung, Beschaffung, Fertigung und Inbetriebnahme des Datenloggers müssen daher mit geringen Zeit- und Kostenaufwand möglich sein.

- **A4: Geringe Masse und Größe**

Um höchste Leistungsdichte und Energieeffizienz zu erreichen, ist Leichtbau das Kernprinzip des modernen Fahrzeugbaus. In der Formula Student gilt dies in besonderem Maße, denn auf das Fahrzeug wirken enorme Quer- und Längsbeschleunigungen, und zusätzliche Masse erhöht nach dem zweiten Newtonschen Gesetz die erforderliche Kraft und damit die physikalische Arbeit für diese Beschleunigung. Der Datenlogger soll den Bauraum und die Fahrdynamik deshalb möglichst wenig beeinträchtigen.

- **A5: Zukunftsfähigkeit**

Systemkonzepte, technische Anforderungen sowie regulatorische Rahmenbedingungen sind in hohem Maße dynamisch. Es bedarf daher eines leicht verständlichen und flexibel anpassbaren Systems, das sowohl bisherige Lösungen ersetzen als auch mit wachsenden Anforderungen skalieren kann.

2.2. Anforderungsmatrix

Die in Abschnitt 2.1 beschriebenen Nutzeranforderungen werden im Folgenden in konkrete technische Anforderungen überführt. Diese müssen dem SMART- Prinzip folgen, das heißt, sie müssen Spezifisch, Messbar, Attraktiv, Realistisch, und Terminierbar sein. [3]

Projektanforderung	Technische Anforderung
A1: Einfache Nutzbarkeit	T1: Standardisierte Datenauslese Die Messdaten müssen über eine Standardschnittstelle auf gängigen Betriebssystemen ohne proprietäre Software auslesbar sein.
A1: Einfache Nutzbarkeit	T2: Offenes Datenformat Die Messdaten müssen in einem offenen, klar dokumentierten Format vorliegen, das mit frei verfügbarer, plattformübergreifender Software analysierbar ist.
A2: Zuverlässige Datenerfassung	T3: Echtzeitfähige Datenerfassung Das System muss CAN-FD-Nachrichten bei der vorgesehenen maximalen Buslast mit deterministischen Latenzen und ohne Nachrichtenverlust erfassen.
A2: Zuverlässige Datenerfassung	T4: Ausfallsichere Speicherung Messdaten müssen dauerhaft gespeichert werden; ein plötzlicher Spannungsverlust darf weder Dateien beschädigen noch die laufende Aufzeichnung unabgeschlossen beenden.

Projektanforderung	Technische Anforderung
A2: Zuverlässige Datenerfassung, A5: Zukunftsfähigkeit	T5: Mehrkanal-Datenerfassung Das System muss mindestens vier unabhängige CAN-FD-Busse gleichzeitig erfassen können und für weitere Datenkanäle erweiterbar sein.
A3: Zeit- und ressourceneffiziente Fertigung	T6: Einfache, schnelle Entwicklung Hard- und Software müssen schnell entwickelt, in Betrieb genommen validiert werden. Dafür sind gut dokumentierte Komponenten, verfügbare Entwicklungswerkzeuge und einfache Debug-Möglichkeiten einzusetzen.
A3: Zeit- und ressourceneffiziente Fertigung	T7: Prüfgerechte Fertigung Hardware, Bauteile und Schnittstellen müssen mit den im Team verfügbaren Mitteln herstellbar sein. Qualitätssicherungsmaßnahmen müssen Fehler systematisch minimieren.
A4: Geringe Masse und Größe	T8: Kompakte, robuste Bauform Masse und Volumen dürfen die Größenordnung der besten auf dem Markt verfügbaren Fahrzeugdatenlogger nicht überschreiten.
A3: Zeit- und ressourceneffiziente Fertigung, A5: Zukunftsfähigkeit	T9: Modulare Erweiterbarkeit Hard- und Software müssen in austauschbare Funktionsbereiche gegliedert sein; zusätzliche Schnittstellen und zukünftige Fahrzeugfunktionen bis hin zur Integration in die Fahrzeugregelung müssen ohne grundlegende Neukonstruktion integrierbar sein.
A5: Zukunftsfähigkeit	T10: Verwendung moderner Komponenten

Tabelle 2: Anforderungsmatrix

2.3. Stand der Technik

Auf dem Markt existieren nur eine geringe Anzahl an CAN-FD-Datenloggern, die diesen Anforderungen entsprechen.

Datenlogger, die für die Automobilindustrie entwickelt wurden, sind meist als robuste Multi-Bus-Logger mit umfangreicher Konnektivität ausgeführt. Die größten Einschränkungen sind dabei oft die Anzahl der CAN-FD-Kanäle, das Gewicht und Volumen sowie der Anschaffungspreis. Die hier gelisteten Geräte entsprechen am ehesten den Anforderungen dieses Projekts und dienen als technische Referenz.

Die öffentlich einsehbaren Eigenschaften der angebotenen Produkte werden nach folgendem Schema gewertet. Die Eigenschaften sind in allen Tabellen den jeweiligen technischen Anforderungen T1 bis T9 zugeordnet.

Bewertung	Bedeutung
-1	Eigenschaft ist für die Erreichung der Projektanforderungen nach Abschnitt 2.2 hinderlich.
0	Eigenschaft hat keine Relevanz für die Projektziele und wirkt somit weder positiv noch negativ.
1	Eigenschaft ist den Projektzielen zuträglich.

Tabelle 3: Bewertungsskala für Marktanalyse

2.3.1. Influx ReXgen Air



Abbildung 3: Influx ReXgen Air [4]

Technische Anforderung	Eigenschaft	Bewertung
T1	Datenübertragung über LTE statt über USB	-1
T1	Verschlüsselte Datenübertragung	-1
T1	Browserbasierte Bedienoberfläche	0
T4	Fest integrierter eMMC-Speicher mit 32 GB	0
T7	Preis von etwa 1.215 USD	1
T8	Masse von 112 g	1
T8	Volumen von 195 cm ³	1
T9	Open-Source-API	1
T9	Integrierte inertielle Messeinheit	0
T5	Vier unabhängige CAN-Kanäle	1
T9	Zwei digitale und zwei analoge Messeingänge	0
	Summe	4

Tabelle 4: Evaluation des Influx ReXgen Air [4]

2.3.2. Kvaser Memorator Pro 5xHS



Abbildung 4: Kvaser Memorator Pro 5xHS [5]

Technische Anforderung	Eigenschaft	Bewertung
T1	Zentraler D-Sub-Steckverbinder für alle Signale	1
T1	Unterstützung von SocketCAN (Linux Kernel- Treiber)	1
T3	Umfangreiche Filterfunktionen	1
T4	SD-Karte mit wählbarer Speicherkapazität	1
T7	Preis von etwa 3.362 USD	-1
T8	Masse von 159 g	0
T8	Volumen von 242 cm ³	1
T5	Fünf CAN-FD-Kanäle	1
	Summe	5

Tabelle 5: Evaluation des Kvaser Memorator Pro 5 [5]

2.3.3. Vector GL2400



Abbildung 5: Vector GL2400 [6]

Technische Anforderung	Eigenschaft	Bewertung
T1	Auswertung mit Vector Signalizer	1
T1	Drahtlose Datenübertragung über 3G	0
T4	Speicherung auf SSD oder SD-Karte	1
T7	Preis nicht öffentlich angegeben	-1
T8	Masse von etwa 580 g	-1
T8	Volumen von 839 cm ³	-1
T5	Vier CAN-Kanäle	1
T9	Unterstützung von Ethernet, FlexRay und LIN	0
	Summe	0

Tabelle 6: Evaluation des Vector GL2400 [6]

Fazit

Der Kvaser Memorator Pro 5xHS überzeugt durch sein geringes Gewicht und einfacher Anbindung von 5 CAN- FD Kanälen, weshalb sein Design im Folgenden analysiert und als technische Referenz genutzt wird.

2.4. Referenzdesign-Analyse

Auf Produktbildern des Memorator Pro 5- PCBs ist erkennbar, dass dieser zum Erreichen von T4 eine 3.7V 500mAh Li-ion Batterie verwendet. Als User-interface wird eine Micro-USB Buchse eingesetzt (T1).



Abbildung 6: PCB des Kvaser Memorator Pro 5 [7]

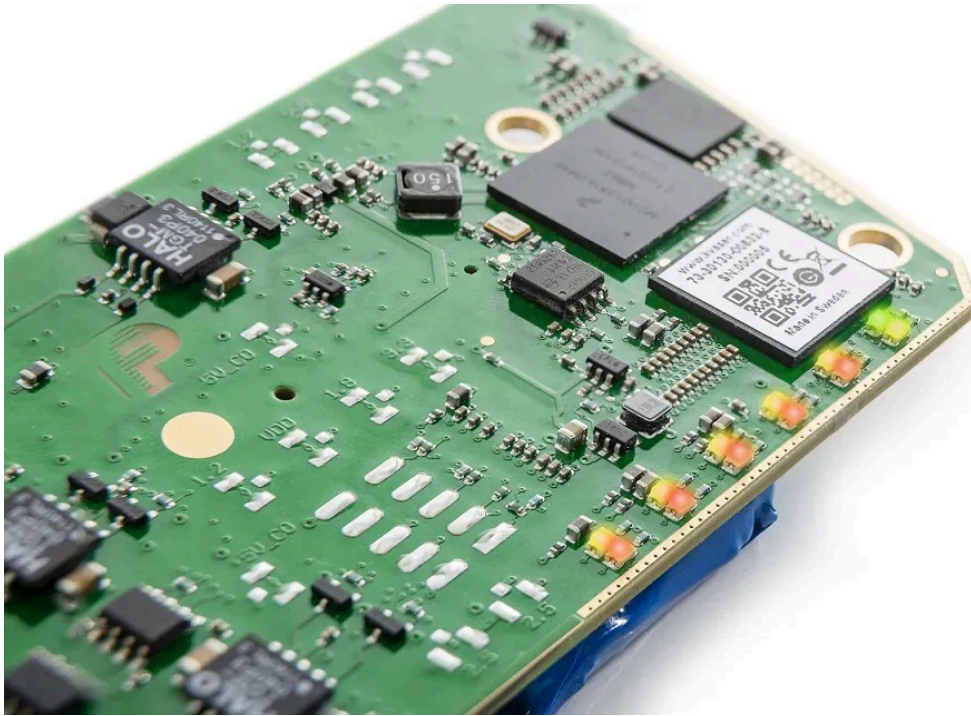


Abbildung 7: PCB des Kvaser Memorator Pro 5 [7]



Abbildung 8: USB- Anschluss am Kvaser Memorator Pro 5 [7]

Anhand von Produktbildern des Memorator Pro 5 ist nicht erkennbar, welche integrierten Schaltkreise genutzt werden. Ein Hinweis findet sich allerdings in der minimaleren und günstigeren Ausführung des Datenloggers, dem Memorator Pro 2, welcher nur 2 CAN-FD Kanäle besitzt.

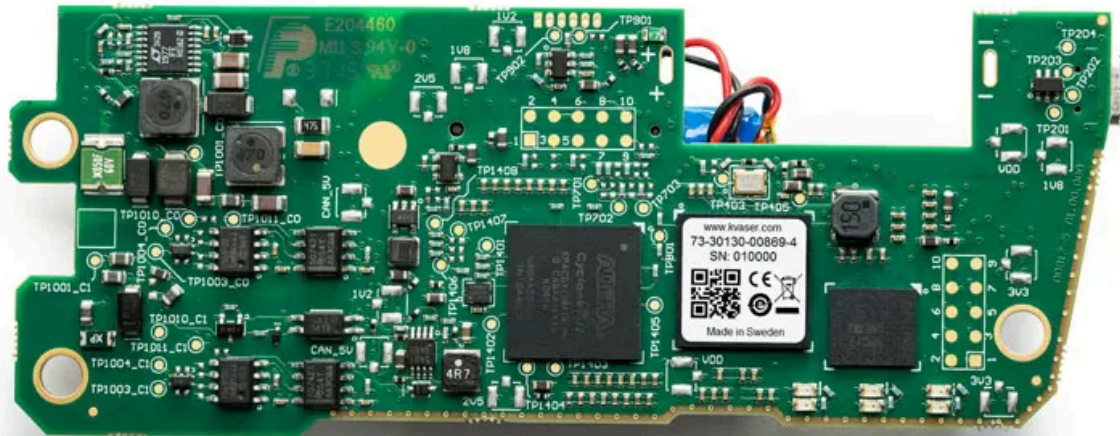


Abbildung 9: PCB des Kvaser Memorator Pro 2 [8]

Bei dieser Platine wurde sich offensichtlich für ein Altera Cyclone- FPGA entschieden. Dies ist technisch eine sehr naheliegende Lösung, da die extrem flexible Gestaltung der digitalen Schaltungen durch individuelle Auswahl von IP-Cores in einem FPGA sich besonders für spezialisierte und hoch Echtzeit- performante Anwendungen eignet.

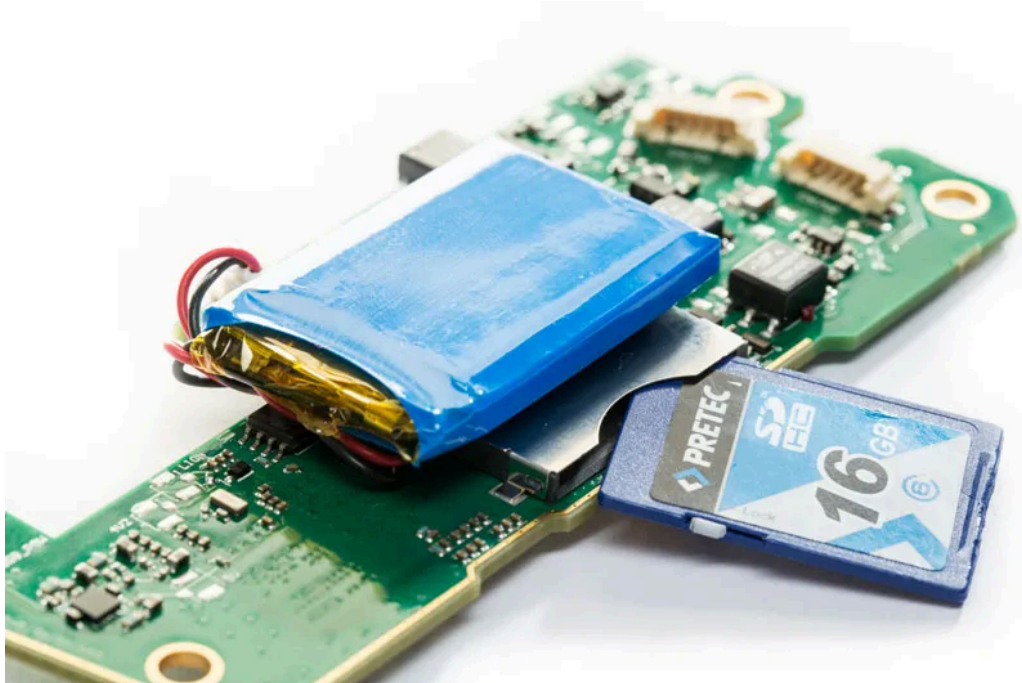


Abbildung 10: PCB des Kvaser Memorator Pro 2 mit Akku und SD-Karte [8]

2.5. Ziele und Umfang

Ziel dieser Studienarbeit ist die Erforschung der grundlegenden Technologien, Auswahl von Komponenten, Architektur für die Entwicklung, eines CAN-FD-Fahrzeugdatenloggers für ein Formula- Student- Fahrzeug.

Hierzu wird ein vereinfachter Prototyp entwickelt, der die zentralen Funktionen eines solchen Systems validiert: das Empfangen von CAN-FD-Nachrichten und das dauerhafte Speichern dieser Daten auf einem lokalen Speichermedium.

Das Konzept gilt als valide, wenn nachgewiesen werden kann, dass der Datenlogger von einem externen Sender mit einer Geschwindigkeit von 1 Mbit/s in der Arbitationsphase und 2 Mbit/s in der Datenphase CAN-FD Frames über einen längeren Zeitraum Verlustfrei abspeichert.

2.6. Meilensteinplanung

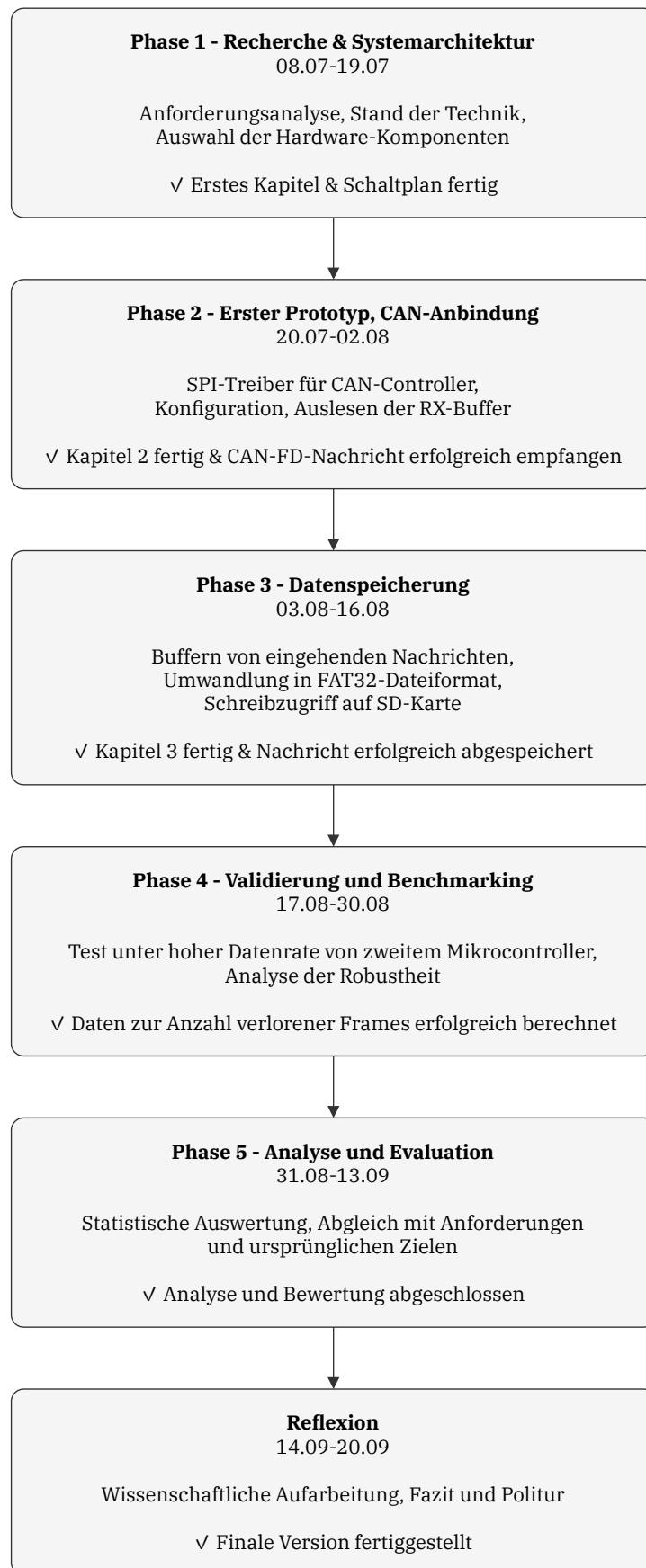


Abbildung 11: Meilensteinplanung

3. Systemarchitektur

3.1. Grundarchitektur

Der Datenlogger wird als modularer Datenpfad aus CAN-FD-Controllern, Mikrocontroller mit USB- Schnittstelle und microSD-Karte aufgebaut. Jede Komponente übernimmt dabei eine klar abgegrenzte Aufgabe. Das reduziert den Entwicklungsaufwand (T6), erleichtert die prüfgerechte Fertigung (T7) und erlaubt spätere Erweiterungen ohne grundlegende Neukonstruktion (T9).

3.1.1. Mikrocontroller

Der Mikrocontroller koordiniert die Peripherie, verarbeitet empfangene Nachrichten und steuert die Datenspeicherung. Gegenüber einer FPGA-Lösung bietet er eine besser zugängliche Entwicklungsumgebung, Debug-Möglichkeiten und verfügbare Softwarebibliotheken; damit unterstützt er die schnelle Entwicklung (T6). Ausreichende Rechenleistung, RAM und flexible Schnittstellen sind erforderlich, um den Datenstrom mit deterministischen Latenzen zu verarbeiten (T3) und mehrere externe Controller sowie das Speichermedium anzubinden (T5).

3.1.2. Externe CAN-FD-Controller

Die Forderung nach mindestens vier unabhängigen CAN-FD-Bussen (T5) kann nicht von einem einzelnen, üblichen Mikrocontroller mit integrierten CAN-FD-Controllern erfüllt werden. Im niedrigen Preissegment bietet aktuell jediglich der STM32G474 drei CAN-FD-Controller. [9] Aus diesem Grund wird jeder Bus über einen spezialisierten externen CAN-FD-Controller angebunden. Dessen Filter und Empfangspuffer sollen den Mikrocontroller entlasten und unterstützen eine verlustfreie Echtzeiterfassung (T3). Die Anzahl der Kanäle kann durch zusätzliche gleichartige Controller erweitert werden (T9).

3.1.3. microSD-Karte

Die Messdaten werden auf einer entnehmbaren microSD-Karte in einem FAT32-Dateisystem gespeichert. Das Standardmedium kann ohne Spezialhardware am PC gelesen werden und unterstützt damit die standardisierte Datenauslese (T1); ein offen dokumentiertes Logformat soll die Auswertung somit einfach gestalten (T2). Das integrierte Flash-Management der Karte sowie ein RAM-Zwischenspeicher entkoppeln Datenerfassung und Schreibvorgang und tragen zur ausfallsicheren Speicherung bei (T4).

3.1.4. USB-Schnittstelle und Platine

USB dient als Standardschnittstelle für Programmierung, Debugging und die direkte Datenauslese (T1, T6). Die Leiterplatte trennt Mikrocontroller, CAN-FD-Anbindung, Speicher und Spannungsversorgung in klar prüfbare Funktionsbereiche (T7, T9). Gut lötbare Gehäuse, Testpunkte und ein für die Fertigung geeignetes Layout

vereinfachen Inbetriebnahme und Qualitätssicherung (T6, T7); eine kompakte Platzierung reduziert zugleich Masse und Volumen (T8).

3.1.5. Sender

Um den Prototypen sinnvoll testen zu können, wird das USB- zu CAN-FD Interface Candlelight FD des Herstellers Linux Automation GmbH verwendet. Dieses basiert seinerseits auf einem STM32G0 und bietet die einfachste Möglichkeit, Nachrichten mit inkrementell angepasstem Inhalt zu versenden, um schlussendlich die Vollständigkeit des Datensatzes zu überprüfen.

Die nachfolgenden Komponentenauswahlen vergleichen zunächst harte Ausschlusskriterien und bewerten die verbleibenden Optionen anschließend anhand der technischen Anforderungen.

3.2. Auswahl des Mikrocontrollers

Entscheidend für die Auswahl sind neben Rechenleistung und RAM für die Echtzeitverarbeitung (T3) vor allem mehrere flexible Schnittstellen für CAN-FD-Controller und Speichermedium (T5). Hohe Priorität haben außerdem eine gut dokumentierte Software-Infrastruktur und einfache Debug-Möglichkeiten (T6), ein fertigungsgerechtes Gehäuse (T7) sowie Reserven für spätere Erweiterungen (T9). In der folgenden Tabelle wird eine Auswahl der aktuellsten Modelle der gängigsten Hersteller gelistet.

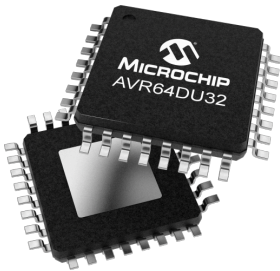
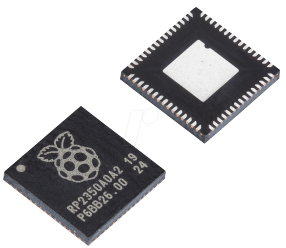
Mikrocontroller	AVR64DU [10] 	STM32-C5 [11] 	RP2350 [12] 
Hersteller	Microchip	STMicroelectronics	Raspberry Pi
Veröffentlichung	2026	2026	2024
Architektur	8-Bit AVR-Mega	32-Bit Cortex-M33	2x 32-Bit Cortex-M33 + 2x 32-Bit RISC-V
Anzahl Prozessoren	1	1	2
RAM-Größe	8 KB	bis 256 KB	520 KB
Schnittstellen	1x SPI, 1x I ² C, 2x USART, USB FS	USB, OctoSPI, CAN-FD	2x SPI, 2x I ² C, USB, 12x PIO-SM
Pin-Multiplexer	PORTMUX eingeschränkt	Alternate-Function-Matrix	sehr flexibel über GPIO-Funktionen
Anzahl CAN-FD-Controller	0	2	0
Treiber/ Software workflow	Melody	STM- HAL, CubeMX-2	Pico C/ C++ SDK

Tabelle 7: Gängige aktuelle Mikrocontroller

Entscheidungsmatrix Mikrocontroller

Im folgenden werden die ausgewählten Mikrocontroller mit einer dreistufigen Bewertung versehen, anschließend wird diese je nach Relevanz für diesen konkreten Anwendungsfall mit einem Multiplikator gewichtet.

Bewertung	Bedeutung
-1	Keine oder ungeeignete Umsetzung des Kriteriums
0	Ausreichende Erfüllung des Kriteriums
1	Besonders vorteilhafte Erfüllung des Kriteriums

Tabelle 8: Bewertungsskala

Multiplikator	Bewertungsrelevanz
1	Grundlegende Relevanz für die Systemauswahl
2	Hoher Einfluss auf die Eignung im Anwendungsfall
3	Entscheidender Einfluss auf die technische Umsetzbarkeit

Tabelle 9: Gewichtungsskala

T-Nr.	Eigenschaft	Gewichtung	AVR64DU [10]	STM32C5 [11]	RP2350 [12]
T9	Modernität	1	1	1	0
T3	Single core- Performance	1	0	1	1
T3	Multi- Core- Architektur	1	-1	-1	1
T3	RAM-Größe	3	-1	0	1
T5	Flexibilität Schnittstellen	2	0	0	1
T7	Flexibilität Signal- Routing	2	0	0	1
T5	CAN-FD- Integration	1	-1	1	-1
T6	Qualität Software- Support	3	0	0	1
	Gewichtete Summe		-4	2	11

Tabelle 10: Entscheidungsmatrix Mikrocontroller

Fazit

Der RP2350 eignet sich vorallem durch seine flexiblen pin-multiplexer, die freie Gestaltung von seriellen Schnittstellen mithilfe von Programmable- Input- Output State Machines (PIO-SM), großem SRAM und Dual-Core RISC-V Prozessoren, sowie aufgrund seiner übersichtlichen, gut dokumentierten C/C++ Entwicklungsumgebung.

Für die schnelle Prototypenentwicklung steht die Entwicklungsplatine Raspberry Pico 2 zur Verfügung.

3.3. Auswahl eines CAN- FD Controller-Transcievers

Für die Anbindung der externen CAN-FD-Busse werden Controller . Besonders relevant sind ein integrierter Transceiver, die maximale Datenrate, die Filtermöglichkeiten und die Größe des internen Nachrichtenspeichers.

Baustein	TCAN4550-Q1 [13]	MCP251863 [14]	MCP2518FD [15]
Hersteller	Texas Instruments	Microchip	Microchip
Transceiver integriert	Ja	Ja	Nein
CAN-FD-Datenrate	8 Mbit/s	8 Mbit/s	8 Mbit/s
SPI-Takt	bis 18 MHz	bis 20 MHz	bis 20 MHz
Nachrichtenspeicher	2 KB	2 KB	2 KB
Filter	128 Standard-ID oder 64 Extended-ID Filter	32 flexible Filter-/Masken	32 flexible Filter-/Masken
FIFO-Struktur	konfigurierbare Rx-/Tx-FIFOs und Tx-Queue	31 konfigurierbare FIFOs und Tx- Queue	31 konfigurierbare FIFOs und Tx- Queue

Tabelle 11: Verfügbare CAN-FD-Controller-Transceiver

T-Nr.	Eigenschaft	Multiplikator	TCAN4550-Q1 [13]	MCP251863 [14]	MCP2518FD [15]
T9	Modernität	1	0	1	0
T7	Simple externe Beschaltung	2	1	1	-1
T3	Hohe CAN-Datenrate	2	1	1	1
T3	Hohe SPI-Datenrate	2	0	1	1
T3	Großer Datenspeicher	3	0	0	0
T3	Effektivität der Filterung	2	0	1	1
T6	Simplizität des Interfaces	3	0	1	0
T7	Geringe Kosten	1	-1	0	1
	Gewichtete Summe		3	12	5

Tabelle 12: Entscheidungsmatrix CAN-FD-Controller

Der MCP251863 bietet eine integrierte Lösung, welche RAM, CAN-Controller sowie Bus-Transceiver auf einem Chip vereint. Seine übersichtliche Registerstruktur, schnelles SPI-Interface und flexible Filterobjekte machen ihn besonders geeignet. Der CAN-Controller des MCP251863 ist mit dem des MCP2518-FD identisch, der einzige Unterschied besteht darin, dass der MCP2518-FD keinen internen CAN-Transceiver besitzt. Für die Entwicklung eines Prototypen wird aufgrund einfacher Verfügbarkeit ein fertiges Modul verwendet, welches einen MCP2518-FD und einen ATA6563-Transceiver kombiniert. Dies ist Softwaretechnisch vollständig äquivalent.

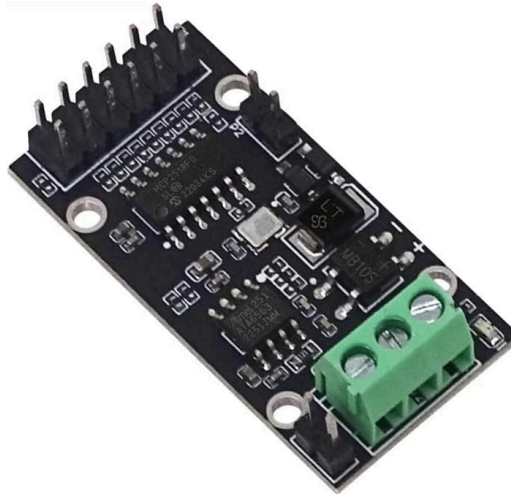


Abbildung 15: MCP-2518-FD Modul.[16]

3.4. Auswahl eines Speichermediums

Für die dauerhafte Speicherung der Messdaten werden SPI-NAND-Flash, eMMC und microSD betrachtet. Entscheidend sind dabei nutzbare Speicherkapazität, Schreibgeschwindigkeit, Schnittstellenaufwand, integriertes Flash-Management, mechanische Integration und die einfache Auslesbarkeit am PC.

Speichermedium	SPI-NAND-Flash [17]	eMMC [18]	microSD [19]
Beispiel	Winbond W25N01GV	Kingston eMMC 5.1	Kingston Industrial microSD
Schnittstelle	SPI / Quad-SPI	eMMC 5.1 HS400	SD / SPI-Modus
Schreibgeschwindigkeit	seitenweise, controllerabhängig	hoch, host- und typabhängig	bis 80 MB/s
Flash-Controller	nein, Host verwaltet Wear-Leveling	ja, ECC und Wear-Leveling integriert	ja, ECC und Wear-Leveling integriert
Auslesbarkeit am PC	nur über eigene Firmware	nur über Adapter/ Testhardware	direkt über Kartenleser
Gehäuse / Integration	SOIC/WSO, gut lötlbar	BGA, schwer zu löten	Socket oder Push-Push-Halter
Verfügbarkeit	gut als Bauteil	gut, aber Variantenbindung	sehr gut als Standardmedium
Kosten pro GB	hoch	mittel	niedrig
Bewertung	robust, aber hoher Softwareaufwand	technisch stark, aber aufwendig zu fertigen	einfach auslesbar und prototypenfreundlich

Tabelle 13: Vergleich verfügbarer Speichermedien

T-Nr.	Eigenschaft	Multiplikator	SPI-NAND-Flash [17]	eMMC [18]	microSD [19]
T4	Nutzbare Speicherkapazität	1	0	0	1
T3	Schreibgeschwindigkeit ³	3	0	1	1
T6	Schnittstellen Komplexität	2	0	-1	1
T4	Integriertes Flash-Management	3	-1	1	1
T1	Auslesbarkeit am PC	3	-1	0	1
T7	Mechanische Integration	2	1	-1	1
T9	Verfügbarkeit	1	0	0	1
T7	Geringe Kosten	2	-1	0	1
	Gewichtete Summe		-6	2	17

Tabelle 14: Entscheidungsmatrix Speichermedium

Die microSD-Karte erzielt die höchste gewichtete Bewertung. Sie verbindet eine direkte PC-Auslesbarkeit mit integriertem Flash-Management, geringem Integrationsaufwand und ausreichender Schreibgeschwindigkeit und wird daher als Speichermedium eingesetzt.

3.5. Blockdiagramm

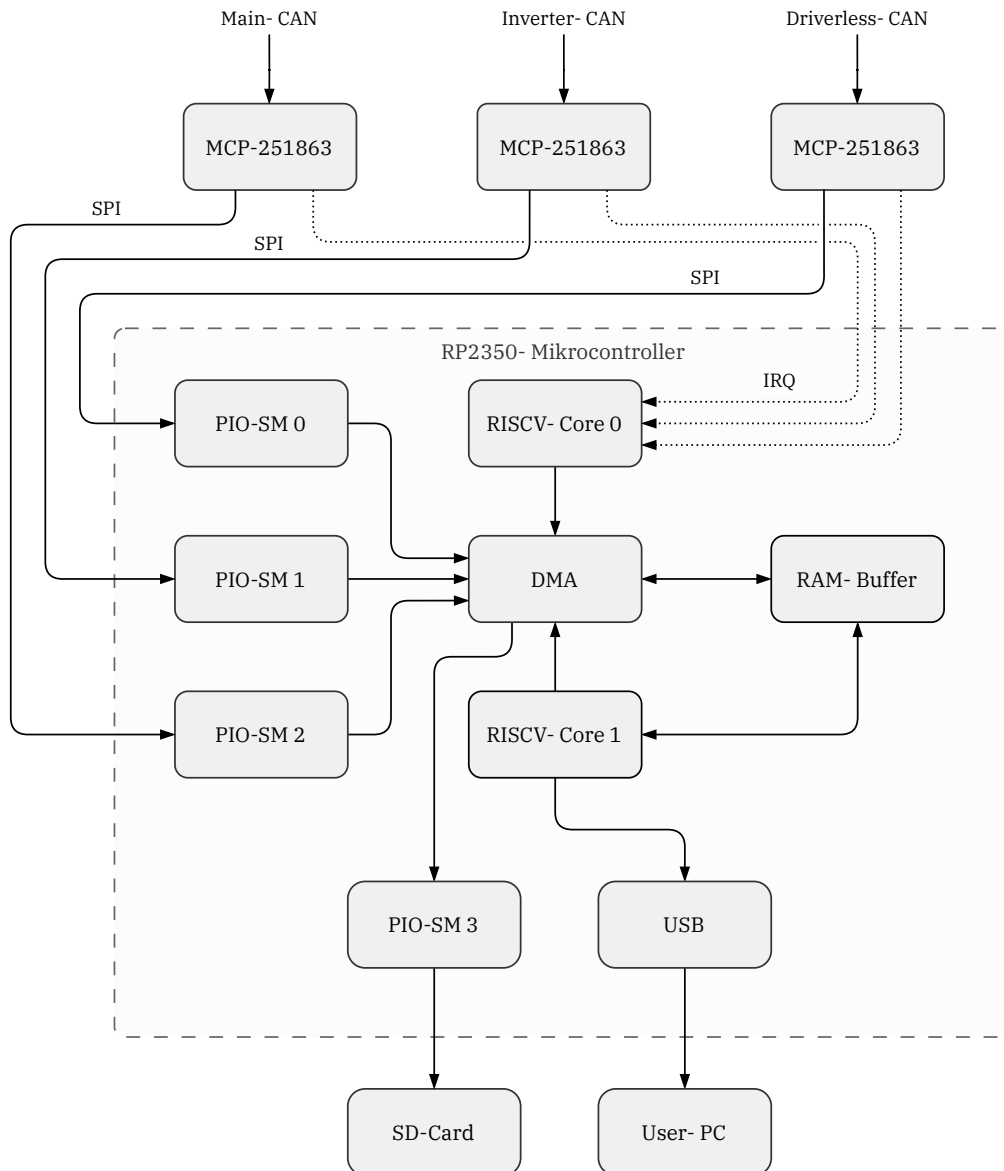


Abbildung 16: Vollständiger Datenlogger.

3.6. Blockdiagramm des Prototypen

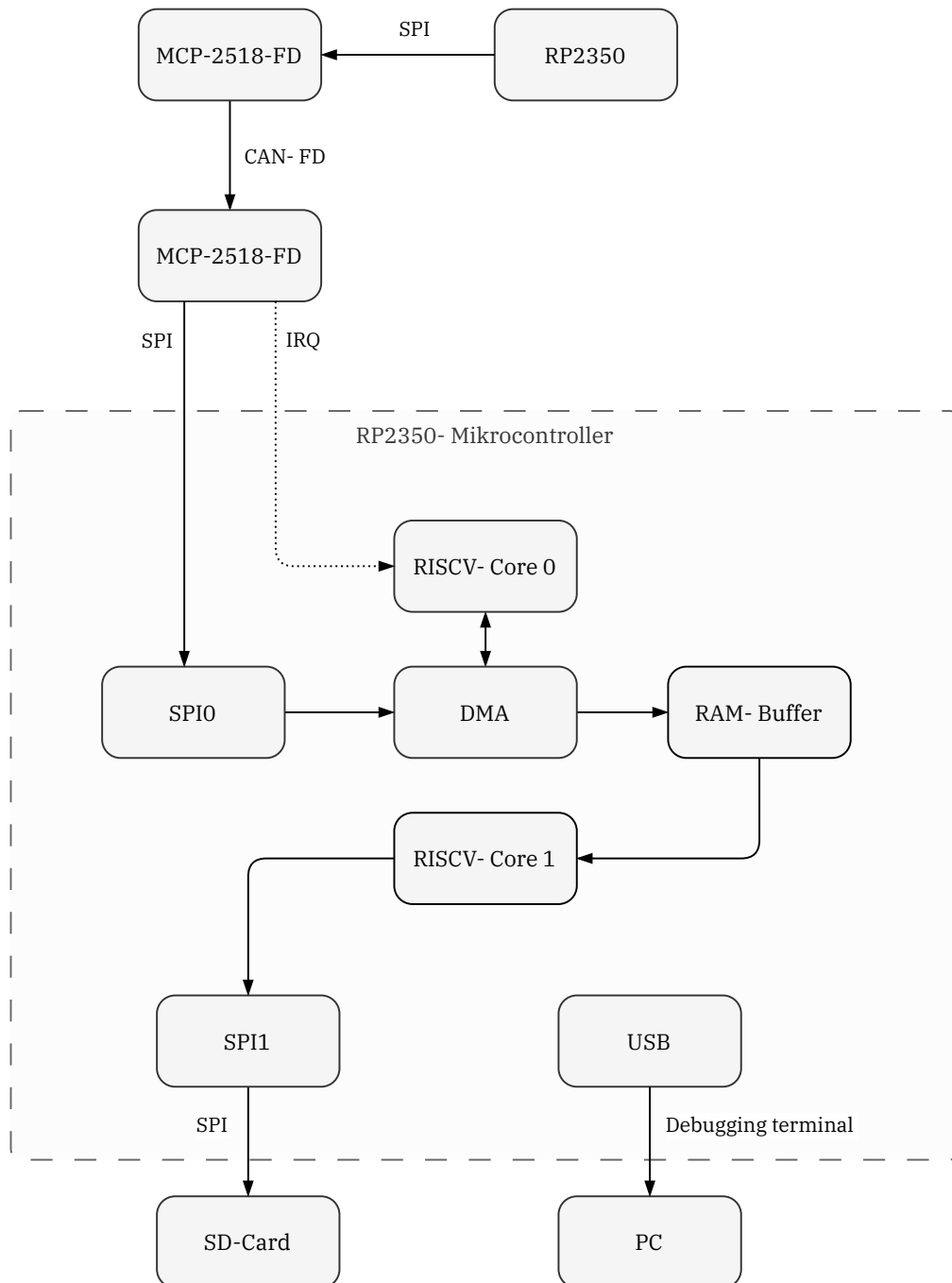


Abbildung 17: Zeitplanung

4. Senden von CAN-FD Nachrichten

Der Candlelight FD bietet den Vorteil, dass dieser vom Linux-Kernel mithilfe seines integrierten Treibers Socketcan als natives Netzwerkinterface erkannt wird.

```
anon@archlinux ~ > ip link show
```

...

```
5: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen
10
    link/can
```

Anschließend kann über `ip link` der Modus und Datenraten eingestellt werden:

```
anon@archlinux ~ >sudo ip link set can0 type can bitrate 1000000 dbitrate 2000000 fd
on
anon@archlinux ~ >sudo ip link set can0 up
```

Um nun eine Nachricht zu senden, wird das freie Programm `canutils` verwendet, welches den Befehl `cansend` bereitstellt:

```
cansend can0 123##1DEADBEEF
```

5. Empfangen von CAN-FD-Nachrichten

5.1. Der Treiber des MCP2518FD

Der RP2350 verarbeitet CAN-FD nicht selbst, sondern über den externen Controller MCP2518FD. Der Treiber trennt deshalb die SPI-Übertragung von CAN-spezifischen Konfigurationen und Nutzdaten.

5.1.1. Serial Peripheral Interface

Das Serial Peripheral Interface (SPI) ist ein voll- Duplex Master-Slave Datenbus. Das bedeutet, es werden Daten immer gleichzeitig vom Master an den Slave und vom Slave an den Master zurückgesendet. Dazu besitzt er folgende Leitungen:

- **CSn**: Chip-Select
- **SCK**: Serial Clock
- **MISO**: Master In, Slave out
- **MOSI**: Master Out, Slave In

Das schalten der Csn-Leitung auf Masse (Active-Low) signalisiert den Beginn einer Übertragung.

5.1.2. Registerzugriff über SPI

Jedes Register des MCP2818FD ist 32 Bit breit und besitzt eine 12-Bit-Startadresse. Der Treiber unterscheidet die drei grundlegenden Befehle zum Schreiben, Lesen und Zurücksetzen.

```
// MCP command bits
#define MCP_COMMAND_WRITE 0b00100000
#define MCP_COMMAND_READ 0b00110000
#define MCP_COMMAND_RESET 0b00000000
```

Da SPI immer mit Byte- Arrays arbeitet [20], muss ein solches für einen erfolgreichen SPI- transfer zunächst aus dem Befehl sowie der Zieladresse im Adressraum des

MCP2518FD zusammengesetzt werden. Das erste Befehlsbyte enthält den vier Bit breiten SPI-Befehl sowie die oberen vier Bits der Registeradresse. Das zweite Byte überträgt die verbleibenden acht Bits der Adresse. Anschließend werden die eigentlichen Daten gesendet. [15]

```
void mcp_write_reg(uint16_t address, void *tx_buffer, size_t length){
    printf("Attempting MCP2518 Register modification... ");

    uint8_t command[2];
    command[0]=MCP_COMMAND_WRITE | (address >>8);
    command[1]=address & 0b000011111111;
    gpio_put(pin_can0_cs, 0);
    sleep_us(1);
    spi_write_blocking(spi_port_can0, command, 2);
    spi_write_blocking(spi_port_can0, tx_buffer, length);
    sleep_us(1);
    gpio_put(pin_can0_cs, 1);

    printf("done.\n");
}
```

Beim Lesen eines Registers des MCP2518FD wird zunächst ebenfalls 2 Bytes aus Befehl und Zieladresse gesendet. Da SPI Vollduplex ist, werden anschließend so viele Nullbytes gesendet, wie von der Zieladresse an Daten gelesen werden sollen, um den Bus zu takten. Gleichzeitig werden die vom MCP2518-FD gesendeten Bytes im Empfangspuffer gespeichert.

```
void mcp_read_reg(uint16_t address, void *rx_buffer, size_t length){
    printf("Attempting MCP2518 Register retrieve... ");

    uint8_t command[2];

    command[0]= MCP_COMMAND_READ | (address>>8);    //4 command bits + first 4
address bits
    command[1]= address & 0b000011111111;    //last 8 address bits

    gpio_put(pin_can0_cs, 0);
    spi_write_blocking(spi_port_can0, command, 2);    //send 2 entries of command
array
    spi_read_blocking(spi_port_can0, 0b00000000, rx_buffer, length); //send empty
bytes and write recieved data in buffer
    gpio_put(pin_can0_cs, 1);

    printf("done.\n");
}
```

Damit die 32 Bit eines Registers sowohl byteweise über SPI als auch feldweise im Programm nutzbar sind, werden sie als Union abgebildet. data_array ist das Übertragungsformat, da die SPI- Treiber des mikrocontrollers immer mit 8-Bit Arrays arbeiten, und bits ordnet den Datenfeldern ihre eigentliche Bedeutung zu.

```
typedef union {
    uint8_t data_array[4];
    struct __attribute__((packed)) {
```

```
uint32_t SJW : 7;           // Synchronization Jump Width bits
uint32_t unimplemented1 : 1;
uint32_t TSEG2 : 7;         // Time Segment 2 bits
uint32_t unimplemented2 : 1;
uint32_t TSEG1 : 8;         // Time Segment 1 bits
uint32_t BRP : 8;           // Baud Rate Prescaler bits
} bits;
} MCP_C1NBTCFG_t;           // Nominal Bit Time Configuration Register
```

5.1.3. Konfiguration des Empfangspfads

Nach einem Reset beginnt die Initialisierung mit den Bitzeiten der nominalen CAN-Phase (1 Mbit/s) und der Datenphase (2 Mbit/s). Die Felder BRP, TSEG1, TSEG2 und SJW bestimmen dabei den Systemtakt- Prescaler und den Abtastzeitpunkt, sowie die maximal zulässige Zeitspanne, mit der der MCP2518 seine eigene Zeitbasis anhand der Datenflanken auf dem CAN-Bus synchronisieren darf.

```
//nominal data rate: 500kbit/s
//sample point: 80%
//system clock: 20MHz
MCP_C1NBTCFG_t mcp_clnbtcfg = {
    .bits = {
        .SJW= 3,           //4 TQ allowed sample point adjustment to synchronize bus
        .TSEG2= 7, // 8 time quanta after sample
        .TSEG1= 30, //31 time quanta before sample
        .BRP= 0           //system clock prescaler for can-controller
    },
};

//data phase rate to 2Mbit/s
MCP_C1DBTCFG_t mcp_cldbtcfg = {
    .bits = {
        .SJW= 1,
        .TSEG2= 1,
        .TSEG1= 6,
        .BRP= 0
    },
};
```

Das Register FIFO 1 wird als Empfangspuffer für genau eine CAN-FD-Nachricht mit bis zu 64 Byte Nutzdaten eingerichtet. TFNRFNIE legt fest, das bei vorhandenen Daten im FIFO ein Interrupt ausgelöst werden soll.

```
MCP_C1FIFOCON_t mcp_clfifocon1 = {
    .bits = {
        .PLSIZE= 0b111, //64 byte payload
        .TFNRFNIE= 1, // Recieve-Fifo not empty Interrupt enabled.
    },
};
```

Filter 0 leitet zunächst alle Nachrichten in FIFO 1. Zusätzlich schaltet RXIE den globalen Empfangsinterrupt ein.

```
MCP_C1INT_t mcp_clint = {
    .bits = {
```

```

        .RXIE = 1, //recieve message interrupt enable
    }
};

MCP_C1FLTCON_t mcp_clfltcon = {
    .bits = {
        .FLTEN0= 1, //enable filter 0
        .F0BP= 1 //save hits in fifo 1
    }
};

```

Die vorbereiteten Register- Objekte werden anschließend jeweils vollständig geschrieben. Erst der letzte Zugriff fordert den Normalbetrieb im CAN-FD-Modus an.

```

mcp_write_reg(MCP_REG_C1NBTCFG, mcp_clnbtcfg.data_array, sizeof
mcp_clnbtcfg.data_array);
    mcp_write_reg(MCP_REG_C1DBTCFG, mcp_cldbtcfg.data_array, sizeof
mcp_cldbtcfg.data_array);
    mcp_write_reg(MCP_REG_C1FIFOCON1, mcp_clfifocon1.data_array, sizeof
mcp_clfifocon1.data_array);
    mcp_write_reg(MCP_REG_C1INT, mcp_clint.data_array, sizeof mcp_clint.data_array);
    mcp_write_reg(MCP_REG_C1FLTCON0, mcp_clfltcon.data_array, sizeof
mcp_clfltcon.data_array);
    mcp_write_reg(MCP_REG_C1CON, mcp_clcon.data_array, sizeof mcp_clcon.data_array);

```

5.1.4. Inbetriebnahme durch den Mikrocontroller

Vor dem ersten Zugriff initialisiert der Mikrocontroller SPI und hält Chip Select auf High. Ein Reset setzt den MCP2518FD in einen definierten Ausgangszustand.

```

// SPI initialisation
spi_init(spi_port_can0, 1000*1000);
spi_set_format(spi_port_can0, 8, SPI_CPOL_0, SPI_CPHA_0, SPI_MSB_FIRST);
gpio_set_function(pin_can0_sck, GPIO_FUNC_SPI);
gpio_set_function(pin_can0_tx, GPIO_FUNC_SPI);
gpio_set_function(pin_can0_rx, GPIO_FUNC_SPI);
gpio_set_function(pin_can0_cs, GPIO_FUNC_SIO);
//chip select as active-low
gpio_set_dir(pin_can0_cs, GPIO_OUT);
gpio_put(pin_can0_cs, 1);

```

Der Reset-Befehl wird bei aktivem Chip Select übertragen. Nach einer kurzen Wartezeit werden die zuvor vorbereiteten Registerwerte geschrieben und der Controller in den CAN-FD-Betrieb versetzt.

```

void mcp_reset(){
    printf("Resetting MCP2518... ");
    uint8_t command[2];
    command[0]= 0b00000000;
    command[1]= 0b00000000;
    gpio_put(pin_can0_cs,0);
    sleep_us(1);
    spi_write_blocking(spi_port_can0,command, 2);
    sleep_us(1);
    gpio_put(pin_can0_cs,1);
}

```

```
    printf("done.\n");  
}
```

```
mcp_reset();  
    sleep_ms(5);  
    mcp_init();
```

5.1.5. Interruptgesteuertes Auslesen

Eine empfangene Nachricht zieht die Interrupt- Leitung des MCP2518FD auf Low. Die Interrupt-Routine führt selbst keine SPI-Übertragung aus, sondern setzt nur ein Flag. So bleibt sie kurz und der eigentliche Zugriff erfolgt kontrolliert in der Hauptschleife.

```
void can0_irq(uint gpio, uint32_t event_mask){  
    printf("Entering CAN-0 interrupt service routine...");  
    mcp0_pending= 1;  
}
```

Der zugehörige GPIO wird als Eingang mit Pull-up konfiguriert und auf den Low-Pegel überwacht.

```
//interrupt from can0 controller  
    gpio_set_function(pin_can0_irq, GPIO_FUNC_SIO);  
    gpio_set_dir(pin_can0_irq, GPIO_IN);  
    gpio_pull_up(pin_can0_irq);  
    gpio_set_irq_callback(can0_irq);  
    gpio_set_irq_enabled(pin_can0_irq, GPIO_IRQ_LEVEL_LOW, 1);
```

Beim gesetzten Flag liest die Callback-Funktion zunächst aus C1FIF0UA1 den Offset der Nachricht im Message-RAM. Anschließend wird das vollständige Nachrichtenobjekt gelesen. Das Setzen von UINC bestätigt die Verarbeitung und bewegt den FIFO-Zeiger auf die nächste Nachricht.

```
void can0_callback(){  
    printf("Entering CAN-0 recieve callback... ");  
  
    //Retrieve the address offset of the recieved CAN-Message stored in message ram  
    from C1FIF0UA1  
    uint32_t offset_tmp = 0;  
    mcp_read_reg(MCP_REG_C1FIF0UA1, &offset_tmp, sizeof offset_tmp);  
  
    //Read this message address and write into CAN-FD message buffer.  
    can_message_object_t can_rx_buffer;  
    mcp_read_reg(offset_tmp + MCP_RAM_BASE, can_rx_buffer.data_array, sizeof  
can_rx_buffer.data_array);  
  
    //set UINC bit in C1FIFOCON1 to prompt load operation of next message offset  
    into C1FIF0UA1  
    MCP_C1FIFOCON_t clfifcon1_tmp;  
    mcp_read_reg(MCP_REG_C1FIFOCON1, clfifcon1_tmp.data_array, sizeof  
clfifcon1_tmp.data_array);  
    clfifcon1_tmp.bits.UINC= 1;  
    mcp_write_reg(MCP_REG_C1FIFOCON1, clfifcon1_tmp.data_array, sizeof  
clfifcon1_tmp.data_array);  
  
    //reset pending flag
```

```
mcp0_pending = 0;
printf("done.\n");
printf("Recieved Message ID: %d", can_rx_buffer.can_message.SID);
}
```

Die Hauptschleife verbindet beide Schritte: Sie verarbeitet eine Nachricht erst, wenn die Interrupt-Routine dies angefordert hat.

```
while(1){
    if(mcp0_pending) can0_callback();
}
```

6. Literaturverzeichnis

- [1] Raceyard, „Raceyard Newsletter - September 2025“. Zugegriffen: 10. Juli 2026. [Online]. Verfügbar unter: <https://raceyard.de/newsletter/>
- [2] VDI, „VDI/VDE 2206- Entwicklung mechatronischer und cyber-physischer Systeme“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <https://www.vdi.de/mitgliedschaft/vdi-richtlinien/programme-zu-vdi-richtlinien/vdi-2206>
- [3] B. für Digitales und Staatsmodernisierung, „SMART-Methode“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: https://www.orghandbuch.de/Webb/OHB/DE/OrganisationshandbuchNEU/4_MethodenUndTechniken/Methoden_A_bis_Z/SMART_Regel_Methode/SMART_Regel_Methode_node.html
- [4] Influx Technology, „ReXgen Air CAN Bus Logger“. Zugegriffen: 7. Juli 2026. [Online]. Verfügbar unter: <https://www.influxtechnology.com/rexgen-air-can-bus-logger>
- [5] Kvaser, „Kvaser Memorator Professional 5xHS“. Zugegriffen: 7. Juli 2026. [Online]. Verfügbar unter: <https://kvaser.com/product/kvaser-memorator-professional-5xhs/>
- [6] Vector Informatik GmbH, „GL Logger“. Zugegriffen: 7. Juli 2026. [Online]. Verfügbar unter: <https://www.vector.com/int/en/products/products-a-z/hardware/data-logger/gl-logger/>
- [7] Kvaser, „Kvaser Memorator Pro 5xHS CB“. Zugegriffen: 11. Juli 2026. [Online]. Verfügbar unter: <https://kvaser.com/product/kvaser-memorator-pro-5xhs-cb/>
- [8] Kvaser, Zugegriffen: 7. Juli 2026. [Online]. Verfügbar unter: <https://kvaser.com/product/kvaser-memorator-pro-2xhs-v2-cb-2/>
- [9] STMicroelectronics, „STM32G4 Datasheet“. Zugegriffen: 14. Juli 2026. [Online]. Verfügbar unter: <https://www.st.com/resource/en/datasheet/stm32g474cb.pdf>
- [10] Microchip Technology Inc., „AVR64DU28/32 Complete Data Sheet“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ProductDocuments/DataSheets/AVR64DU-28-32-DataSheet-DS40002548.pdf>
- [11] STMicroelectronics, „STM32C5 Series“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: <https://www.st.com/en/microcontrollers-microprocessors/stm32c5-series.html>
- [12] Raspberry Pi Ltd, „RP2350“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: <https://www.raspberrypi.com/products/rp2350/>
- [13] Texas Instruments, „TCAN4550-Q1 Automotive Controller Area Network Flexible Data Rate System Basis Chip with Integrated Controller and Transceiver“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: <https://www.ti.com/lit/gpn/tcan4550-q1>

- [14] Microchip Technology Inc., „MCP251863 External CAN FD Controller with Integrated Transceiver“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: <https://ww1.microchip.com/downloads/aemDocuments/documents/APID/ProductDocuments/DataSheets/MCP251863-External-CAN-FD-Controller-with-Integrated-Transceiver-DS20006624.pdf>
- [15] Microchip Technology Inc., „MCP2518FD External CAN FD Controller with SPI Interface“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: <https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ProductDocuments/DataSheets/External-CAN-FD-Controller-with-SPI-Interface-DS20006027B.pdf>
- [16] Lastdaytime, „SPI to CANFD MCP2518FD Module“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: https://www.amazon.de/dp/B0FY36G29K?ref=ppx_yo2ov_dt_b_fed_asin_title
- [17] Winbond Electronics Corporation, „W25N01GV 1G-Bit Serial SLC NAND Flash Memory“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <https://www.winbond.com/resource-files/w25n01gv%20revh%20082420.pdf>
- [18] Kingston Technology, „Kingston eMMC Product Flyer“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: https://media.kingston.com/pdfs/emmc/emmc_flyer_en.pdf
- [19] Kingston Technology, „Industrial microSD Memory Card Datasheet“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: https://www.kingston.com/datasheets/SDCIT2_en.pdf
- [20] Raspberry Pi Foundation, „Pico C SDK/ Hardware APIs“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: https://www.raspberrypi.com/documentation/pico-sdk/hardware.html#group_hardware_spi

7. Tabellenverzeichnis

Tabelle 1		6
Tabelle 2	Anforderungsmatrix	7
Tabelle 3	Bewertungsskala für Marktanalyse	8
Tabelle 4	Evaluation des Influx ReXgen Air [4]	8
Tabelle 5	Evaluation des Kvaser Memorator Pro 5 [5]	9
Tabelle 6	Evaluation des Vector GL2400 [6]	10
Tabelle 7	Gängige aktuelle Mikrocontroller	19
Tabelle 8	Bewertungsskala	19
Tabelle 9	Gewichtungsskala	20
Tabelle 10	Entscheidungsmatrix Mikrocontroller	20
Tabelle 11	Verfügbare CAN-FD-Controller-Transceiver	21
Tabelle 12	Entscheidungsmatrix CAN-FD-Controller	22
Tabelle 13	Vergleich verfügbarer Speichermedien	24
Tabelle 14	Entscheidungsmatrix Speichermedium	25

8. Abbildungsverzeichnis

Abbildung 1	Der A-25 von Raceyard. [1]	3
Abbildung 2	V-Modell. [2]	4
Abbildung 3	Influx ReXgen Air [4]	8
Abbildung 4	Kvaser Memorator Pro 5xHS [5]	9
Abbildung 5	Vector GL2400 [6]	10
Abbildung 6	PCB des Kvaser Memorator Pro 5 [7]	11
Abbildung 7	PCB des Kvaser Memorator Pro 5 [7]	12
Abbildung 8	USB- Anschluss am Kvaser Memorator Pro 5 [7]	12
Abbildung 9	PCB des Kvaser Memorator Pro 2 [8]	13
Abbildung 10	PCB des Kvaser Memorator Pro 2 mit Akku und SD-Karte [8]	13
Abbildung 11	Meilensteinplanung	16
Abbildung 12		19
Abbildung 13		19
Abbildung 14		19
Abbildung 15	MCP-2518-FD Modul.[16]	23
Abbildung 16	Vollständiger Datenlogger.	26
Abbildung 17	Zeitplanung	27